

Ethers.js Wallet Integration — AI Agent Skill

A guide for AI coding agents to integrate ethers.js v6 into a React frontend for wallet connection, network management, and smart contract interaction.

Overview

This skill teaches an AI agent how to integrate [ethers.js v6](https://docs.ethers.org/v6/) into a React project. The agent should:

1. Read the user's project structure (paths to smart contract artifacts, backend configs)
2. Fetch the relevant ethers.js docs to verify patterns
3. Generate production-ready code

Source-Driven Development

Every framework-specific code decision **MUST** be backed by official documentation.

Source: <https://docs.ethers.org/v6/getting-started/>

Do not implement from memory — ethers.js v6 has significant API differences from v5. Always verify against the docs at `https://docs.ethers.org/v6/`.

Key v6 API changes from v5

v5	
new ethers.providers.Web3Provider(window.ethereum)	new ethers.Bro
provider.getSigner() returns JsonRpcSigner directly	await prov Promise<Js
ethers.utils.parseEther	ethers.par
ethers.utils.formatEther	ethers.for
ethers.utils.parseUnits	ethers.par
ethers.utils.formatUnits	ethers.for
BigNumber class	Native bigin
contract.connect(signer)	contract.c but runner typ

User Project Structure

The agent should ask the user for paths to key project directories. If the user does not supply paths, scan the working directory for common patterns:

```
project-root/
├── frontend/                                # React app root
│   ├── src/
│   │   ├── hooks/                         # Custom React hooks
│   │   ├── components/                   # UI components
│   │   ├── lib/                          # Utilities, ABIs, configs
│   │   └── App.tsx
│   └── package.json
├── contracts/                              # Solidity source
│   ├── src/
│   ├── out/                              # Foundry artifacts (ABIs + by
│   │   └── MyContract.sol/
│   │       └── MyContract.json
│   └── artifacts/                        # Hardhat artifacts
```

```

├── ┌── deployments/      # Deployed addresses
│   └── broadcast/      # Foundry broadcast logs
└── backend/           # Optional backend
    ├── src/
    └── .env
└── .env                # Root env file

```

What to ask the user

1. Path to React frontend (e.g. `./frontend`)
2. Path to smart contracts directory (e.g. `./contracts`)
3. Path to deployed contract addresses (e.g. `./contracts/deployments/fuji/MyContract.json`)
4. Which network(s) to support (e.g. Fuji testnet, Sepolia, mainnet)
5. Path to ABI files (usually `out/` for Foundry or `artifacts/` for Hardhat)

If the user supplies none, scan the environment:

- Look for `package.json` with "ethers" dependency (check the version — v6+ required)
- Look for `contracts/` or `smart-contracts/` directories
- Look for `artifacts/` or `out/` directories
- Look for `.env` files with `NEXT_PUBLIC_*` or `VITE_*` vars containing contract addresses
- Look for deployment JSON files in `broadcast/`, `deployments/`, or similar

Implementation Guide

Step 1: Install ethers

```

npm install ethers
# or
bun add ethers

```

```
# or
pnpm add ethers
```

Verify the installed version is v6+. Check `package.json` after install.

Step 2: Create a wallet connection hook

Create `src/hooks/use-wallet.ts` (or equivalent in the project).

Pattern (source: <https://docs.ethers.org/v6/getting-started/#starting-connecting>):

```
import { useState, useEffect, useCallback } from "react"
import { BrowserProvider, JsonRpcSigner } from "ethers"

export function useWallet() {
  const [address, setAddress] = useState<string | null>(null)
  const [signer, setSigner] = useState<JsonRpcSigner | null>(null)
  const [isConnecting, setIsConnecting] = useState(false)
  const [error, setError] = useState<string | null>(null)

  const connect = useCallback(async () => {
    if (!window.ethereum) {
      setError("Install MetaMask or another wallet")
      return
    }
    setIsConnecting(true)
    setError(null)
    try {
      const provider = new BrowserProvider(window.ethereum)
      const accounts: string[] = await provider.send("eth_accounts", [])
      if (accounts.length === 0) throw new Error("No accounts found")
      const s = await provider.getSigner()
      setAddress(accounts[0])
      setSigner(s)
    } catch (err) {
      setError(err instanceof Error ? err.message : "Failed to connect")
    } finally {
      setIsConnecting(false)
    }
  }, [setError, setSigner, setAddress])

  return { connect, address, signer, isConnecting, error }
}
```

```

        setIsConnecting(false)
    }
}, [])

const disconnect = useCallback(() => {
    setAddress(null)
    setSigner(null)
    setError(null)
}, [])

useEffect(() => {
    if (!window.ethereum) return
    const handleAccountsChanged = (accounts: string[]) => {
        if (accounts.length === 0) {
            disconnect()
        } else {
            setAddress(accounts[0])
        }
    }
    window.ethereum.on("accountsChanged", handleAccountsChanged)
    return () => {
        window.ethereum.removeListener("accountsChanged", handleAccountsChanged)
    }
}, [disconnect])

return { address, signer, isConnected, error, connect, disconnect }

```

EIP-6963 multi-wallet support (optional, recommended): Source: <https://docs.ethers.org/v6/api/providers/#BrowserProvider-discover>

```

// Modern approach using EIP-6963 discovery
import { BrowserProvider } from "ethers"

async function connectEIP6963() {
    const provider = await BrowserProvider.discover()
    if (!provider) throw new Error("No wallet detected")
    const accounts = await provider.send("eth_requestAccounts")
    const signer = await provider.getSigner()
}

```

```
    return { provider, signer, address: accounts[0] }  
  }  
}
```

Step 3: Ethereum type declarations

Create `src/types/ethereum.d.ts` for the `window.ethereum` type:

```
interface Window {  
  ethereum?: import("ethers").Eip1193Provider & {  
    on?: (event: string, cb: (...args: any[]) => void) =>  
      removeListener?: (event: string, cb: (...args: any[])  
  }  
}
```

Step 4: Network configuration

Create `src/lib/networks.ts`:

```
export interface NetworkConfig {  
  chainId: number  
  chainIdHex: string  
  rpcUrl: string  
  name: string  
  symbol: string  
  explorerUrl: string  
}  
  
export const NETWORKS: Record<string, NetworkConfig> = {  
  fuji: {  
    chainId: 43113,  
    chainIdHex: "0xa869",  
    rpcUrl: "https://api.avax-test.network/ext/bc/C/rpc",  
    name: "Avalanche Fuji",  
    symbol: "AVAX",  
    explorerUrl: "https://testnet.snowtrace.io",  
  },  
}
```

```

sepolia: {
  chainId: 11155111,
  chainIdHex: "0xaa36a7",
  rpcUrl: "https://rpc.sepolia.org",
  name: "Sepolia",
  symbol: "ETH",
  explorerUrl: "https://sepolia.etherscan.io",
},
}

```

Switch network using the injected provider's

wallet_switchEthereumChain RPC:

```

async function switchNetwork(chainIdHex: string) {
  if (!window.ethereum) throw new Error("No wallet")
  try {
    await window.ethereum.request({
      method: "wallet_switchEthereumChain",
      params: [{ chainId: chainIdHex }],
    })
  } catch (err: any) {
    // 4902 = chain not added yet
    if (err.code === 4902) {
      throw new Error("Chain not added. Call addNetwork f
    }
    throw err
  }
}

```

```

async function addNetwork(config: NetworkConfig) {
  if (!window.ethereum) throw new Error("No wallet")
  await window.ethereum.request({
    method: "wallet_addEthereumChain",
    params: [{
      chainId: config.chainIdHex,
      chainName: config.name,
      nativeCurrency: { name: config.symbol, symbol: conf
      rpcUrls: [config.rpcUrl],
      blockExplorerUrls: [config.explorerUrl],
    }
  })
}

```

```
    }],  
  })  
}
```

Step 5: Smart contract integration

Locating the ABI

The agent should find the ABI from the user's contract artifacts:

```
Foundry: contracts/out/MyContract.sol/MyContract.json → p  
Hardhat: contracts/artifacts/contracts/MyContract.sol/MyC  
Truffle: contracts/build/contracts/MyContract.json → pars
```

If the user has no artifacts directory, the agent should ask for the ABI or suggest compiling the contracts first.

Locating the contract address

Check these locations in order:

1. `contracts/deployments/{network}/MyContract.json`
— contains address field
2. `contracts/broadcast/` — Foundry broadcast logs contain `contractAddress`
3. `.env` files — variables like `TOKEN_ADDRESS`, `CONTRACT_ADDRESS`, `NEXT_PUBLIC_TOKEN_ADDRESS`
4. User's backend `.env` or config files

Creating the contract hook

Pattern (source: <https://docs.ethers.org/v6/getting-started/#starting-contracts>):

```
import { useMemo } from "react"  
import { Contract } from "ethers"
```

```
import type { JsonRpcSigner, Provider } from "ethers"
import MyContractABI from "@lib/abis/MyContract.json"

const CONTRACT_ADDRESS = process.env.NEXT_PUBLIC_CONTRACT_ADDRESS

export function useContract(signer: JsonRpcSigner | null, provider: Provider) {
  const contract = useMemo(() => {
    const runner = signer ?? provider
    if (!runner || !CONTRACT_ADDRESS) return null
    return new Contract(CONTRACT_ADDRESS, MyContractABI, runner)
  }, [signer, provider])

  return contract
}
```

Key points:

- Connect to a `Signer` for write operations (transactions)
- Connect to a `Provider` for read-only operations (queries, static calls)
- Use `useMemo` to avoid re-creating the contract on every render
- The `Contract` class is a meta-class — methods and events are derived from the ABI at runtime

Read-only calls (view/pure)

```
// Source: https://docs.ethers.org/v6/getting-started/#static-calls
const balance = await contract.balanceOf(address)
const symbol = await contract.symbol()
const decimals = await contract.decimals()

// Format for display
import { formatUnits } from "ethers"
const displayBalance = formatUnits(balance, decimals)
```

State-changing calls

```
// Source: https://docs.ethers.org/v6/getting-started/#st
const tx = await contract.transfer(to, amount)
const receipt = await tx.wait()
console.log("Transaction mined:", receipt.hash)
```

Simulating a transaction (staticCall)

```
// Source: https://docs.ethers.org/v6/getting-started/#st
// Useful for preflighting without spending gas
const result = await contract.transfer.staticCall(to, amo
```

Error handling for contract calls

```
try {
  const tx = await contract.someMethod(arg)
  await tx.wait()
} catch (err: any) {
  // Ethers v6 wraps revert reasons
  if (err.code === "CALL_EXCEPTION") {
    console.error("Revert reason:", err.reason)
  } else if (err.code === "ACTION_REJECTED") {
    console.error("User rejected the transaction")
  } else {
    console.error("Transaction failed:", err)
  }
}
```

Common error codes:

- `CALL_EXCEPTION` — contract reverted
- `ACTION_REJECTED` — user rejected in wallet
- `INSUFFICIENT_FUNDS` — not enough native tokens for gas
- `UNPREDICTABLE_GAS_LIMIT` — gas estimation failed
- `BUY_ETHER_UNWRAP_FAILED` — ENS related
- `REPLACEMENT_UNDERPRICED` — replacement tx gas too low

Step 6: Reading contract events

```
// Source: https://docs.ethers.org/v6/getting-started/#st

// Listen for events
contract.on("Transfer", (from, to, amount, event) => {
  console.log(`${from} -> ${to}: ${formatUnits(amount, 18)}`)
})

// One-time listener
contract.once("Transfer", (from, to, amount, event) => {
  console.log("First transfer detected!")
})

// Query historic events
const filter = contract.filters.Transfer(null, address) /
const events = await contract.queryFilter(filter, -1000)
```

Step 7: Token balances and formatting

```
// Source: https://docs.ethers.org/v6/getting-started/#st
import { formatEther, parseEther, formatUnits, parseUnits }

// User input → wei
const amountInWei = parseEther("1.5") // 1500000000000000000
const amountInGwei = parseUnits("10", "gwei") // 10000000000000000000

// wei → display string
formatEther(amountInWei) // "1.5"
formatUnits(amountInGwei, "gwei") // "10.0"

// Token balance with custom decimals
const balance = await contract.balanceOf(address) // bigint
const decimals = await contract.decimals() // bigint
const display = formatUnits(balance, decimals) // string
```

Step 8: Environment variables

Place contract addresses and config in `.env.local` (Next.js) or `.env` (Vite):

```
# Next.js (prefix with NEXT_PUBLIC_)
NEXT_PUBLIC_TOKEN_ADDRESS=0x...
NEXT_PUBLIC_GAME_MASTER=0x...
NEXT_PUBLIC_API_URL=http://localhost:3001

# Vite (prefix with VITE_)
VITE_TOKEN_ADDRESS=0x...
```

Project-Specific Patterns

If the user has a Foundry project

- ABI path: `contracts/out/{ContractName}.sol/{ContractName}.json` — parse `.abi` field
- Deployment addresses: `contracts/broadcast/{ContractName}.sol/{chainId}/run-latest.json` — parse `transactions[0].contractAddress`
- The agent should read these files and extract the ABI + address

If the user has a Hardhat project

- ABI path: `contracts/artifacts/contracts/{ContractName}.sol/{ContractName}.json` — parse `.abi` field
- Deployment addresses: `contracts/deployments/{network}/{ContractName}.json`

If the user has no contracts directory

The agent should ask the user for the ABI and contract address. The user may have:

- A deployed contract they don't have the source for
- The ABI from a third-party protocol (e.g. Uniswap, OpenSea)
- Environment variables already set up

Files the agent should create/modify

1. `src/lib/networks.ts` — Network configurations
2. `src/hooks/use-wallet.ts` — Wallet connection hook
3. `src/types/ethereum.d.ts` — Type declarations
4. `src/lib/abis/` — ABI JSON files (copy from contracts directory)
5. `.env.local` — Add `NEXT_PUBLIC_*` env vars (do NOT commit secrets)

Verification checklist

- ☐ `ethers` is installed and version is v6+
- ☐ `window.ethereum` check before using `BrowserProvider`
- ☐ `provider.getSigner()` is awaited (returns `Promise` in v6)
- ☐ ABI is human-readable JSON array format (not the full Solidity JSON output)
- ☐ Contract address is read from env var or config, never hardcoded (except known public addresses)
- ☐ Network switching uses `wallet_switchEthereumChain/` `wallet_addEthereumChain`
- ☐ `BigInts` are formatted with `formatUnits/formatEther` before display
- ☐ `window.ethereum.on("accountsChanged", ...)` listener is registered
- ☐ Cleanup: listener removal in `useEffect` return
- ☐ No secrets or private keys in frontend code
- ☐ Error handling covers user rejection (`ACTION_REJECTED`) and

contract reverts (CALL_EXCEPTION)